Attesting Outputs and Delegation Ancestry in Multi-Agent AI Systems

Lifei Liu, Haoran Yu
Independent Researcher
Seattle, WA, USA
lliu.lifei@gmail.com, haoranyu889@gmail.com
Corresponding author: Lifei Liu

arXiv:2608.30387v1 [cs.CR] 31 Aug 2026

***Abstract*—Multi-agent applications delegate work across independently operated deployers. After an incident, a verifier must answer two questions: which deployer released the reported bytes, and whether each cross-deployer edge was authorized. Credentials establish who may act, but need not bind them to later output bytes or prove both deployers authorized a dynamically created edge.**

We present a two-layer attestation design for dynamic delegation without a shared authority, public log, or precommitted workflow. A trusted deployer runtime signs a hash of each released output; this records released bytes but does not prevent prompt injection. Ancestry evidence records edge authorization. Under a unified threat model, we compare a signed linked list, a Merkle-chain variant, and a co-signed DAG. The primitives are standard; the contribution is deployer-side binding and the evidence needed for the two questions. After child-key compromise, the single-signer designs permit an unauthorized parent binding, whereas the co-signed DAG rejects it because the parent must authorize the edge. Fixed adversary matrices and regression tests validate the composed verifier. On an Apple M1 Pro, ancestry-only checks take 24.3–499.2 μs per hop. In a live local multi-service workflow, a parent discovers the child's A2A Agent Card; the child calls an MCP tool and releases local-LLM output: all 30 signed-DAG tasks passed complete verification, while a controlled child-key-only claim was rejected; its mean end-to-end latency was 813.1 ms versus 770.8 ms without evidence. In a complementary three-availability-zone AWS deployment, all 1,000 valid co-signed-DAG paths verified; issuance averaged 3.651 ms and complete verification 5.015 ms. The cloud result excludes TLS/mTLS, KMS, and model-serving latency.

***Index Terms*—AI agent identity, output attestation, delegation ancestry, multi-agent systems, cryptographic verification, cloud security**

I. Introduction

AI agents increasingly complete tasks through multi-hop delegation across deployment boundaries [1], [2]. Consider a deployer that delegates a task to an independently operated agent. After that agent releases an output, incident response must answer two questions: which deployer released these exact bytes, and did the parent deployer authorize this particular cross-deployer edge? Authorization credentials record a principal and permission at issuance time; they need not bind later output to a trusted runtime or establish a complete, authorized path.

These are post-incident evidence questions, not claims about agent behavior. Indirect prompt injection can alter an agent's behavior after an authorization check has succeeded [3]. A signature cannot establish that an output was benign or prevent its production, but it can provide tamper-evident evidence of the bytes emitted and the delegation path for investigation and targeted revocation.

Existing delegation mechanisms identify an actor or grant a permission [4], [5], but, in this setting, leave two evidence gaps for dynamically created paths. *G1 (output binding):* a credential does not attest bytes later released by a runtime. *G2 (edge consent):* under child-key compromise, a child-only signature on a new parent reference does not demonstrate that the named parent approved the relationship.

Our key insight is that these gaps require two independently verifiable evidence layers rather than a stronger credential. A trusted deployer runtime signs a hash of each released output, while an ancestry record carries the authorization evidence for each edge. The verifier must decide from a presented path because the setting may lack a shared authorization service, public log, or precommitted workflow. The primitives are standard; our contribution is the deployer-side binding and the evidence required for this dynamic setting. We compare a signed linked list, a Merkle-chain variant, and a co-signed DAG. After child-key compromise, only the co-signed DAG rejects a new parent claim because it requires the parent's signature on the edge. Figure 1 illustrates the two evidence layers and this distinction.

We make the following contributions:

1) **A deployer-side output-attestation interface.** A deployer runtime, rather than the LLM agent, hashes each attested output and signs the complete record with deployer and validity metadata. The interface records what was released under a trusted-runtime assumption; it does not detect or prevent prompt injection.
2) **A unified analysis of ancestry evidence.** We formulate a single system and threat model for both layers, then evaluate three standard ancestry-binding constructions as comparative designs. The key result is that a signed linked list and a Merkle-chain variant use only the child deployer's authorization for an edge; a co-signed DAG additionally requires the parent deployer's consent and thus resists unauthorized cross-deployer edge creation after child-key compromise.
3) **Implementation and deployment evidence.** We validate the specified predicates with fixed matrices and regression tests. We measure issuance and checking

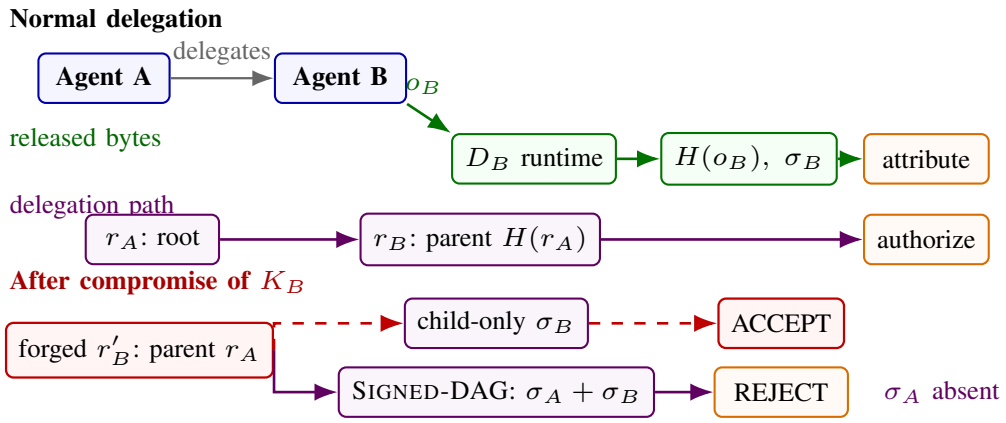


Fig. 1. Motivating example: output receipts and delegation records answer different forensic questions. The deployer runtime binds a receipt to the released bytes; after compromise of D_B's key, a child-only chain accepts a forged parent claim, whereas SIGNED-DAG requires both endpoint signatures and rejects it when σ_A is absent.

cost in LangGraph, process-separated, container, and three-availability-zone AWS deployments, and execute a live local multi-service workflow with A2A JSON-RPC, an MCP Streamable-HTTP tool, and LLM-produced child output. Historical LLM-generated aggregates are exploratory context, not current security evidence.

The result is a bounded deployment guideline: a linked representation is appropriate for a single trust domain, while resistance to unauthorized cross-deployer edges after child-key compromise requires explicit parent authorization. We do not claim that the construction prevents agent compromise; it makes the resulting outputs and delegation path auditable.

II. Background and Related Work

a) Agent authorization and behavioral evidence: Agent-identity work distinguishes authorization from the behavior that follows it. Indirect prompt injection exploits this separation [3], and high probe AUC alone need not establish malicious-content detection [6]. Chan et al. [7] identify behavioral verification as a missing capability. MCP standardizes tool and context exchange, while A2A defines protocol-level authentication and authorization requirements for inter-agent communication [5], [8]. Our deployer-side binding records released bytes; it neither judges semantic safety nor prevents their production.

b) Agent ancestry and delegation protocols: W3C DID and Verifiable Credentials [9], [10] provide single-hop primitives, while the IETF agent-auditing architecture records delegation transitions in a broader interoperable framework [11]. South [4] addresses authenticated delegation of authority to agents. Llambí-Morillas and Fernández-Fernández [12] instead formalize evidence that a concrete request satisfies a policy in an execution context. Our narrower comparison binds released output bytes and identifies the signatures needed for a cross-deployer edge after child-key compromise; it can supply evidence to these authorization and audit layers.

c) General provenance and chain integrity: in-toto [13] validates a precommitted software supply-chain layout, while SPIFFE/SPIRE issues workload identities from registered trust domains and supports their update propagation [14]. Our setting instead creates delegations on demand across independently operated deployers, without a public log or predeclared graph.

TABLE I
COMPARISON WITH CONCURRENT AGENT-IDENTITY SYSTEMS. HERE, "DEPLOYER-SIDE" MEANS THAT A SIGNER SEPARATE FROM THE LLM/AGENT OBSERVES RELEASED SEGMENTS; AIP'S OPTIONAL AGENT-SIGNED FINAL RESULT HASH IS NOT COUNTED.

Property	AIP	HDP	MemLineage	Ours
Deployer-side output binding	✗	✗	✗	✓
Multi-hop chain integrity	✓	✓	✓	✓
Comparative chain analysis	✗	✗	✗	✓
Parent edge co-signature	✗	✗	✗	✓
Child-key-compromise analysis	✗	Excluded	✗	✓
Structural-verifier testing	✗	✗	✗	✓

d) Concurrent agent-identity protocols: AIP [15] uses append-only capability tokens and an optional agent-signed final-result hash. HDP [16] excludes key compromise, MemLineage [17] targets memory integrity, and Mandato [18] targets pre-action policy conformance. Our target is post-incident evidence of released bytes and bilateral deployer authorization of an edge. Table I compares the concrete identity and ancestry proposals.

e) Complementary controls: Information-flow control [19] addresses behavioral compromise. JWT [20] and OAuth [21] express signed claims and delegated permissions, but do not by themselves bind later bytes to a credential or establish a complete ancestry path.

III. System and Unified Threat Model

A. System Model and Security Objectives

We consider a principal that delegates a task through agents operated by deployers $D_A, D_B, \ldots$. A deployer is the service that hosts an agent, controls its signing key, and emits a delegation record when the agent invokes another agent. Each record identifies the issuer, child instance, proposed parent, declared depth, expiry time, nonce, and evidence construction.

TABLE II
UNIFIED TRUST ASSUMPTIONS AND THEIR CONSEQUENCES.

Component	Assumption	Consequence if violated
Agent / LLM	Untrusted	It may emit harmful output; attestation records it.
Deployer runtime	Faithfully signs released bytes	Output evidence no longer reflects runtime output.
Signing key	Secret, except in §VI-B1	A compromised child motivates parent co-signature; both endpoints defeat all designs.
Key registry	Admission-controlled	Self-enrolled identities become indistinguishable from legitimate deployers.
Verifier	Enforces stated predicates	Invalid evidence can be accepted; addressed by tests and red-team mutation.
Transport	Untrusted	Signatures and hashes detect modification.

A consumer receives the records and verifies a claimed path from a root principal to a leaf agent. The prototype has no separately named global `chain_id`; a chain is the ordered, presented root-to-leaf path.

The design produces two kinds of evidence. *Output evidence* binds a deployer-issued credential to bytes released by its runtime. *Ancestry evidence* binds each delegation record to a parent and, for cross-deployer edges, can require both endpoint deployers to approve the relationship. The objectives are therefore: (O1) reject substituted output bytes and replay at a verifier with persistent nonce state; (O2) reject malformed, truncated, or spliced ancestry presented to the consumer; and (O3) under a single child-deployer key compromise, reject a new cross-deployer edge not authorized by its parent. These are post-incident evidence objectives. They do not prevent an LLM from following a malicious instruction, assess whether a signed output is safe, or enforce application-specific authorization policy.

B. Unified Trust Boundary

We trust four components: the deployer runtime to observe and sign the bytes it releases, its private signing key, the registry that admits authorized deployers, and the verifier implementation. Everything outside those components, including the LLM agents, tool responses, network transport, and supplied chain documents, may be malicious. This boundary applies to both layers: Layer 1 relies on the deployer runtime for faithful observation, and Layer 2 relies on the registry and verifier to interpret a delegation record.

The adversary can control tool responses and agent behavior, observe public keys and verifier source, replay or modify evidence in transit, and construct arbitrary structural payloads. It cannot forge a signature under an uncompromised key, find a collision or preimage that defeats the hash binding, compromise the trusted runtime, or enroll an arbitrary deployer in the admission-controlled registry. The single-key compromise study relaxes the secrecy assumption for one deployer's signing key. Table II states the consequence of each trust-boundary violation.

TABLE III
THREAT SCOPE AND CORRESPONDING VERIFIER STATE.

Threat	Status	Mechanism	State
B1–B4 structural edits	In scope	Signature and ancestry predicates	None
B5 sibling fanout	Policy-dependent	Signed limit and child counter	Per parent
Truncation	In scope	Signed ancestry depth	None
Replay	In scope	Nonce uniqueness and expiry	Per verifier
Output substitution	In scope	Signed output hash	Per verifier
Output order / sessions	Not implemented	Needs signed sequence fields	Per session
Canonical encoding	Prototype scope	Deterministic local serializer; not JCS	None
Key registry	Out of scope	Admission control is trusted	—
Endpoint collusion	Out of scope	Both keys compromised	—
Prompt-injection prevention	Out of scope	Requires IFC / monitoring	—

C. Cryptographic Instantiation

Ed25519 and SHA-256 are concrete implementation choices, not architectural requirements. The analysis uses the abstract properties of strong signature unforgeability under chosen-message attack and hash collision resistance (and, where required, preimage resistance). An implementation may substitute equivalent primitives provided it preserves an unambiguous encoding, domain separation, and the same security properties. The prototype uses deterministic Python JSON serialization (recursive key sorting, compact separators, and ASCII escaping), Ed25519 [22], and SHA-256. This serialization is deterministic within the prototype but is *not* a full interoperable implementation of RFC 8785/JCS [23].

D. Structural Attacks and Scope

The adversary's structural goal is to make a verifier accept an ancestry that does not represent the actual delegation. We evaluate four graph-edit classes: orphan insertion (B1), middle-node removal and relinking (B2), transplanting a node across chains (B3), and revising an ancestor after children exist (B4). Sibling fanout (B5) is different: it creates several valid children under a valid parent and therefore requires an application policy such as a signed child limit plus verifier state. Table III distinguishes these cases from threats outside the protocol boundary.

The taxonomy deliberately addresses evidence manipulation, not the attacker's incentive after causing harm. A forged, shortened, or reattached path may not undo an action, but it can misattribute the deployer responsible for that action and obstruct revocation or incident investigation. We make no claim that chain integrity alone deters every prompt-injection attacker.

E. Holistic Security Rationale

An output verifier accepts only when the signed output hash, primary signature, key identifier, validity interval, and nonce

predicate match the bytes it received. An ancestry verifier accepts only when each record has a registered issuer, its parent reference and declared depth are consistent, and any required cross-deployer co-signatures are present. The two checks use separate signed fields but share the trust boundary above: failure of an output check does not satisfy an ancestry predicate, and a structurally valid chain does not authenticate altered output bytes.

Consequently, under the primary model, an accepted tampered output would imply either a failure of the signature/hash assumptions or a violation of the trusted deployer runtime. An accepted structural edit would imply a missing verifier predicate, a cryptographic break, or a compromised signing key. The key-compromise analysis in Section VI-B1 isolates the last case. Adaptive and lifecycle campaigns in Section VI test whether the implementation actually enforces these stated predicates; they do not empirically validate Ed25519 or SHA-256.

F. Registry and Replay Semantics

The verifier maintains a local registry of authorized deployer public keys, analogous to a TLS trust store. The prototype checks that each envelope `key_id` is derived from the pinned public key and can locally revoke that identifier. It does not implement a production key-rotation protocol, durable distributed revocation propagation, or a cross-verifier replay store. A caller can attach a persistent nonce store to reject replay at one verifier instance. These requirements do not introduce a global delegation authority, but they do mean that the protocol does not protect an open ecosystem in which an attacker can self-register a deployer. A signed-DAG edge binds the signed construction identifier, the co-signer set, and the child's nonce in its edge manifest; there is no separate edge-nonce field in the current schema.

IV. Output Attestation (Layer 1)

Output attestation addresses a narrow question: given bytes received by a consumer, can it verify that a particular deployer runtime released those bytes? This is a general cryptographic operation, and we do not claim the signature construction itself as an AI-specific primitive. Its relevance here is placement: the deployer performs the binding after the runtime produces an output and before the output is delivered or delegated. The agent neither holds the signing key nor populates the signed fields.

A. Signed Output Record

For each attested output, the prototype emits a versioned record containing the instance and system identifiers, deployer name, creation and expiry times, parent reference, declared ancestry depth, chain proof, SHA-256 `output_hash`, a 128-bit `nonce`, a signed `binding_scope` label, lifecycle metadata, and optional policy and attribute fields. The wire envelope additionally carries a public-key-derived `key_id`. The deployer primary-signs a deterministic, recursively key-sorted JSON serialization of the complete record with Ed25519.

The composed verifier resolves the deployer in a trusted local registry, checks that the envelope `key_id` matches that registry entry and has not been revoked, verifies the primary signature and validity interval, and recomputes the output hash over the exact bytes supplied by the consumer. It also rejects duplicate nonces in a presented chain and, when a caller supplies a persistent nonce store, a nonce previously consumed by that verifier. These checks establish integrity and origin under the unified trust boundary in Section III; they do not establish that the content is safe or policy-compliant.

B. Current Granularity Boundary

The schema supports the labels `per_session`, `per_turn`, `per_tool_call`, and `per_n_tokens`. They are signed so that an application can state which bytes a record is intended to cover, but the prototype does *not* implement session identifiers, segment indices, tool-call identifiers, or a streaming sequence verifier. Its executable LangGraph evaluation therefore signs one deterministic output per graph node; it is not evidence for streaming or token-window enforcement. A deployment requiring those semantics must add and verify explicit session and sequence fields before making claims about reordering or cross-session transplantation.

C. Scope of the Evidence

The signed record supports a post-hoc audit: an operator can identify the deployer that released a byte sequence and correlate the record with its delegation ancestry. It is compatible with information-flow control, rule-based monitors, or semantic judges, but it does not replace them. If a trusted runtime signs an attacker-directed output, the record faithfully attests to that fact; preventing the output requires a separate control before release.

V. Ancestry Attestation (Layer 2)

Layer 2 asks what evidence a verifier needs to reconstruct a dynamically created delegation path. We start with familiar single-signer constructions as comparative baselines, then add the property required for cross-deployer authorization under child-key compromise. The primitives themselves are standard; the design comparison makes explicit which deployer authorizes each edge and what is lost when that key is compromised.

A. Comparison Criteria

Each node contains the issuer, instance and system identifiers, parent reference, nonce, declared ancestry depth, expiry time, output commitment, and chain-proof fields. We evaluate the constructions against three requirements: (R1) bind a node to its parent so that structural edits B1–B4 are detectable under uncompromised keys; (R2) permit complete-path verification by a consumer without a public log; and (R3) require parent consent when a child under a different deployer claims a cross-deployer parent. R3 is the differentiator in the key-compromise setting.

B. Baseline 1: Signed Linked List

The first baseline adds a signed parent commitment to an otherwise independent per-node signature:

$$\texttt{prev_hash} = \mathrm{SHA256}(\mathsf{canonical_json}(\text{parent}))\,.$$

The issuing deployer signs its complete record, including `prev_hash`. A verifier traverses the presented path, checks each issuer signature and recomputes the commitment. This conventional hash-chain construction satisfies R1 and R2: removing a middle node, transplanting a node, or modifying an ancestor breaks a descendant commitment when all relevant signing keys remain secret. Its verification cost is one hash and one signature check per hop.

The baseline does not satisfy R3. A child deployer can sign a new record that points to any publicly visible parent record. Its `prev_hash` is well-formed, but the parent never approved the new relationship. This is not a hash-chain failure: it is a missing authorization statement from the parent.

C. Baseline 2: Merkle-Chain Variant

The second baseline replaces the per-hop parent commitment with a Merkle commitment [24] over the current ancestry prefix:

$$\texttt{merkle_root} = \mathsf{MerkleRoot}(v_0, \ldots, v_k),$$

where leaves are canonical node representations. This is a standard way to obtain logarithmic-size inclusion proofs when a consumer verifies a selected ancestor rather than the entire path. For the full-path verification required by R2, it adds another consistency check but not a different authorization principal. A compromised child can recompute the affected root and sign the new node, so the construction also fails R3.

We include the Merkle variant because it is a natural alternative often suggested for provenance. It may be useful when a deployment genuinely needs partial-path proofs. That optimization is outside this evaluation, whose consumer verifies the complete ancestry to detect omission and splice attacks.

D. Cross-Deployer Design: Co-Signed DAG

The SIGNED-DAG construction adds an authorization statement for each cross-deployer edge. For an edge from parent node v_A at deployer D_A to child node v_B at deployer D_B, both deployers sign the same domain-separated canonical edge manifest:

$$\bar{v}_B = v_B[\texttt{chain_proof.signatures} \leftarrow \varnothing],$$
$$\tau = \mathsf{Canon}(\texttt{domain}, \mathrm{SHA256}(\mathsf{Canon}(v_A)), \bar{v}_B),$$
$$\sigma_{AB} = \big(\mathrm{Sig}_{sk_A}(\tau), \mathrm{Sig}_{sk_B}(\tau)\big).$$

Thus, τ binds the parent receipt digest and every child field, including the child deployer, output hash, expiry, nonce, lifecycle record, proof design, and co-signer policy. Only the signature list being created is blanked to avoid a signature cycle; the child's primary signature then seals the completed record. The explicit domain tag prevents a valid edge signature from being reinterpreted by another protocol. The child signature attests that D_B accepted the delegation; the parent signature attests that D_A authorized this specific edge. The verifier requires both signatures for cross-deployer edges and checks parent references and declared depth for the rest of the path. Same-deployer edges need only the local deployer's signature.

Attack goal	Linked-List	Merkle	Signed-DAG
Rewrite same-deployer suffix	ACCEPTS	ACCEPTS	ACCEPTS
Bind arbitrary cross-deployer parent	ACCEPTS	ACCEPTS	REJECTS

Child deployer key compromised; entries show verifier outcome.

Fig. 2. Child-key-compromise outcomes. All three designs accept a same-deployer suffix rewrite, whereas only the co-signed DAG rejects an arbitrary cross-deployer parent binding because it requires the uncompromised parent's co-signature.

The property comes with an operational trade-off. The parent deployer must be online when the edge is created, and a refused or timed-out counter-signature causes the delegation to fail closed. A deployment can use the linked representation for same-deployer hops and queue a cross-deployer request. The in-process harness centralizes co-signer access for reproducible tests; the process, container, and AWS experiments separately exercise remote signer RPC.

E. Path Completeness and Lifecycle Checks

Signature and parent checks alone do not ensure that the consumer received a complete path. Each record therefore contains a signed `ancestry_depth`; the verifier rejects a presentation whose number of records differs from the declared depth. Nonce checks prevent reuse of a record at the same verifier.

The implementation represents lifecycle events (`new`, `reload`, `branch`, `composite`, and `fine_tune`). Their semantic invariants are separate from the cryptographic predicates. The current evaluation does not claim a complete lifecycle-semantics audit; a deployment must validate those invariants against its own runtime event model.

VI. EVALUATION

We evaluate three questions. First, what protection does each ancestry construction provide, including under a compromised deployer key? Second, does the implementation enforce the specified structural predicates on fixed and adaptive inputs? Third, what is the cost of issuing and verifying the evidence?

A. Methodology and Platform

The prototype is implemented in Python 3.11 with Ed25519 from `cryptography` ≥ 42 and a deterministic local JSON serializer (not a full RFC 8785/JCS implementation). Unless otherwise noted, experiments ran on a MacBook Pro with



Fig. 3. Fixed-mutation conformance coverage. Across 27,000 mutations, no invalid presentation was accepted. The composed-receipt matrix repeats B1–B4; B5 requires stateful fanout verification and is exercised only in ancestry-only mode. Each check aggregates three designs and 1,000 trials per design.

TABLE IV
MODEL-TO-HARNESS COVERAGE. A ZERO-SUCCESS STRUCTURAL RUN CHECKS IMPLEMENTATION CONFORMANCE, NOT MODEL STRENGTH OR PRIMITIVE SECURITY.

Harness	Driver	Status	Coverage / outcome
Ancestry-only	Python	Current	3 designs × B1–B5 × 1,000; 0 accepted per cell
Composed receipt	Python	Current	3 designs × B1–B4 × 1,000; 0 accepted per cell
Structural	LLMs	Historical	Linked-list-focused exploratory campaign; not rerun after current edge revision
Framework	LangGraph 1.2.6	Current	1,000 deterministic runs; composed verifier accepts all valid chains
Live workflow	MCP + A2A + Ollama	Current	30 tasks/condition; actual tool and LLM bytes; S-DAG verifies every valid task
Process S-DAG	Python `spawn` + Pipe	Current	1,000 valid cross-deployer paths; stale-key reject and fresh-key rotation check
Container S-DAG	Docker bridge + TCP	Current	1,000 valid paths; replay, registry, revocation, and refusal faults fail closed
Cloud S-DAG	AWS EC2 + SSM	Current	1,000 valid paths across three zones; the same four faults fail closed

an Apple M1 Pro CPU (six performance and two efficiency cores), 16 GB unified memory, and macOS 15.7.4. The separate E11–AWS run used three Amazon Linux 2023 EC2 hosts in distinct availability zones; its configuration-specific timings are reported separately below.

For the current revision, we reran a fixed adversary matrix with 1,000 trials for each design–attack pair. The B1–B4 matrix has an ancestry-only mode and a composed-receipt mode; the latter also checks each record's primary signature, `key_id`, expiry, output hash, and nonce predicate. Unexpected verifier exceptions abort the experiment rather than being counted as rejections. Table IV distinguishes these current, deterministic checks from historical LLM-guided exploratory campaigns.

The current matrix is reproducible and measures conformance on a finite, hand-specified attack set; its confidence interval is not a cryptographic or open-world security guarantee. The older LLM-guided aggregate files are retained as exploratory context but are not presented as validation of the revised complete-verifier or signed-edge implementation.

TABLE V
STRUCTURAL OUTCOME UNDER UNCOMPROMISED KEYS. ✓ = ATTACKER CAN CONSTRUCT AN ACCEPTED CHAIN; – = VERIFIER REJECTS.

	NO-CHAIN	HASH-ONLY	NAIVE-SIG	LINKED-LIST	MERKLE	SIGNED-DAG
B1	✓	✓	–	–	–	–
B2	✓	✓	✓	–	–	–
B3	✓	✓	–	–	–	–
B4	✓	✓	–	–	–	–
B5	✓	✓	✓	–†	–†	–†

†Requires a signed `max_children` policy and verifier state.

B. Structural Security Comparison

We first isolate the effect of ancestry evidence. NO-CHAIN carries no inter-node relationship, HASH-ONLY adds an unsigned parent hash, and NAIVE-SIG signs only the node's own identity. The remaining three designs are the comparative constructions in Section V. Table V summarizes the expected verifier outcome for the attack taxonomy under uncompromised keys. In particular, independent node signatures cannot detect a middle-node splice (B2), because every remaining node still verifies on its own. A signed parent commitment is the minimal additional evidence for that case. B5 is policy-dependent. It is retained as a separate stateful fanout-detector condition in the fixed matrix, rather than evidence for a stateless cryptographic ancestry predicate.

1) Child-key compromise: The distinguishing case is a compromised child deployer D_B attempting to claim an arbitrary parent D_A. In the linked and Merkle variants, D_B can compute the public parent commitment and sign the new child record; neither format asks D_A to approve that edge. In the co-signed DAG, the edge transcript also needs D_A's signature. This is the central security difference, rather than a claim that any of the underlying primitives is new.

TABLE VI
KEY-COMPROMISE ANALYSIS.

Compromised party	Attack goal	LL/Merkle	S-DAG
Child D_B	Bind to arbitrary parent	accepts	rejects
Parent D_A	Rewrite parent after child reference	rejects[†]	rejects
Same deployer	Rewrite entire suffix	accepts	accepts
Both endpoints	Forge cross-deployer edge	accepts	accepts

[†]The child's signed parent commitment anchors the original parent.

Definition 1 (Edge-authorization game)**.** *The challenger generates deployer keypairs and gives the adversary one child key sk_i. The adversary may obtain signatures for legitimate edge transcripts from uncompromised deployers. It wins if it creates an accepted cross-deployer edge to an uncompromised parent that did not sign that transcript.*

Theorem 2 (Cross-deployer edge authorization)**.** *For a* SIGNED-DAG *edge, under Ed25519 SUF-CMA security and SHA-256 collision resistance, the probability of winning the edge-authorization game is negligible in the security parameter. For* LINKED-LIST *and* MERKLE*, the probability is one when the compromised party is the child endpoint, because the parent does not sign the edge.*

Proof. An accepted SIGNED-DAG edge contains the uncompromised parent's signature over the complete edge transcript. A new unauthorized transcript therefore yields a signature forgery, except with negligible probability. The two single-signer formats require only the child's valid signature and a correctly computed, public parent commitment, so the compromised child can create the record directly. □ □

Table VI identifies the limits of the result. It proves edge creation only; it does not provide policy-semantic authorization, revocation freshness, or resistance when both endpoints collude.

C. Implementation-Conformance Validation

The current ancestry-only matrix covers three designs, five attack classes (B1–B5), and 1,000 trials per cell; all 15 cells rejected every trial (Wilson 95% upper bound 0.38% per cell). A separate composed-receipt matrix repeats B1–B4 for all three designs, again with zero accepts in all 12 cells. B1–B4 exercise malformed chain presentations; B5 exercises the separate local fanout detector. Figure 3 summarizes test-mode coverage and the common zero-acceptance outcome. This result is not an empirical security bound: under the assumptions of Section III, a successful edit would indicate an omitted or incorrect verifier predicate. Its value is implementation validation.

Archived LLM-guided and lifecycle-mutation aggregates lack prompts, variants, or revision metadata. We retain them as exploratory context only; the current deterministic matrices are the evidence for the revised implementation.

D. Performance and Framework-Integration Overhead

At ancestry depth five over 1,000 trials, mean ancestry-only checks took 24.3 μs for LINKED-LIST, 72.7 μs for MERKLE, and 499.2 μs for SIGNED-DAG. They exclude primary receipt checks and are not complete-verifier costs. The Signed-DAG value includes verification of the revised full-child edge manifest; we do not report a stale serialized-evidence size from the previous edge format.

We also integrated the LINKED-LIST signing hook into a compiled LangGraph 1.2.6 `StateGraph` with three executed nodes (Planner → Researcher → Writer), where the writer runs under a second deployer. Over 1,000 executions after 100 warm-up runs, every three-node chain passed the composed verifier (primary signature, receipt hash, expiry, nonce uniqueness, and linked-list ancestry). Signing averaged 0.171 ms per node (0.214 ms at p95), or 0.512 ms per task (0.663 ms at p95); receipt–chain verification averaged 0.832 ms (1.013 ms at p95). Nodes use deterministic outputs and no LLM client, so these are local protocol-overhead measurements, not model-serving latency; the experiment also omits SIGNED-DAG's online co-signature handshake.

a) Live MCP/A2A/LLM workflow (E12): We next execute the protocol path that the local LangGraph benchmark omits. Three loopback-only processes represent separate deployer trust domains: a parent signer, a child A2A JSON-RPC agent, and an MCP Python-SDK Streamable-HTTP tool server. For each task, the parent discovers the child's Agent Card, invokes A2A `message/send`, and the child calls the MCP `get_evidence` tool before a local Ollama `llama3.1:latest` model (temperature zero) produces the released text. The child signer then returns either no evidence, a child-primary-signed LINKED-LIST receipt, or a SIGNED-DAG receipt after an online parent co-signature over the exact provisional child transcript. The coordinator reconstructs a public-only registry and uses the complete verifier with a fresh file-backed nonce store for each presentation.

After ten warm-up tasks per condition, we ran 30 tasks per condition with a rotated condition order. Table VII reports the entire path, including discovery, A2A, MCP, local LLM, issuance, and complete verification; all conditions use the same fixed tool evidence and prompt. Every attested task passed complete verification and no task aborted. The Signed-DAG mean is 42.3 ms (5.5%) above the no-evidence path; its 11.6 ms child issuance and 16.7 ms complete verification are small relative to its 681.3 ms mean LLM generation. It carries 2,096 bytes of evidence, 228 bytes more than the linked-list condition. This is a task-level measurement, not a claim that local-model output is deterministic or that the LLM is semantically safe.

b) Cross-deployer deployment evidence: E10 (`spawn`/Pipe) and containerized E11 each verified 1,000 two-signer paths beyond the in-process harness; their timing and fault traces are retained in the artifact. We next report the separately configured AWS run.

c) Key deployment result: E11–AWS: We ran E11 on three `t3.small` EC2 hosts in distinct availability zones: parent signer, child signer, and public-only coordinator. Digest-pinned

TABLE VII
E12 LIVE WORKFLOW: 30 TASKS PER CONDITION AFTER TEN WARM-UPS. E2E INCLUDES A2A, MCP, AND LOCAL LLM; ISSUANCE AND VERIFICATION APPLY TO ATTESTED CONDITIONS. THE CONTROLLED ATTACK IS NOT AN ATTACK-SUCCESS-RATE ESTIMATE.

Condition	Valid	E2E mean/p95 (ms)	Evidence (B)	Child-key-only claim
No evidence	30/30	770.8 / 846.0	0	Unauditable
LINKED-LIST	30/30	826.2 / 941.1	1,868	Accepts
SIGNED-DAG	30/30	813.1 / 927.7	2,096	Rejects

containers retained signer keys; SSM managed hosts, and authenticated TCP used private addresses with signer ingress limited to the coordinator security group. After 100 warm-ups and 1,000 trials, every path verified and none aborted. Issuance averaged 3.651 ms (4.384 ms at p95), and complete verification 5.015 ms (6.642 ms at p95). Durable replay, an unregistered parent, and a revoked child key were rejected; parent refusal aborted issuance. These configuration-specific measurements exclude TLS/mTLS, KMS signing, model-client, and production authorization-service latency.

VII. DEPLOYMENT IMPLICATIONS AND LIMITATIONS

The design choice depends on the trust relationship. A signed linked list had the lowest measured local ancestry-verification cost within one deployer trust domain. A Merkle commitment can support partial-path proofs, which we do not evaluate. A co-signed DAG is appropriate when a deployment requires explicit parent authorization for cross-deployer edges and can tolerate an online co-signature handshake.

The LangGraph measurement uses deterministic outputs, while E12 adds a local MCP/A2A workflow with actual local-LLM output. E12's loopback services are separate deployer trust domains, not independent organizations or AWS accounts; it does not measure public-network latency, TLS/mTLS, OAuth, KMS, or production authorization-service latency. Archived LLM-generated aggregates are not current security evidence. A compromised runtime, open registry, cross-verifier replay prevention, production key-rotation protocol, endpoint collusion, streaming-order semantics, LLM quality, and application-policy enforcement remain outside the model.

VIII. CONCLUSION

We have presented deployer-side evidence for outputs and delegation ancestry in multi-agent systems. The core result is that a child-signed parent commitment does not authorize a cross-deployer edge after child-key compromise, whereas a parent co-signature over the complete edge manifest does. The prototype confirms the specified predicates and verifies the co-signed-DAG path in process, container, three-zone AWS, and live local MCP/A2A/LLM workflows. It supports post-incident audit and attribution, not prompt-injection prevention. The anonymous artifact package, including current sources, raw results, manifests, and regression commands, is available at https://anonymous.4open.science/r/cscloud-artifacts-6342/.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," arXiv:2210.03629, 2022.

[2] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.

[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*. ACM, 2023, pp. 79–90.

[4] T. South, "Authenticated delegation and authorized AI agents," arXiv:2501.09674, 2025.

[5] A2A Protocol Working Group, "Agent2agent (A2A) protocol specification," A2A Protocol Specification, 2026. [Online]. Available: https://a2a-protocol.org/latest/specification

[6] Y. Li, Z. Fan, and Z. Zhuang, "When AUC 0.998 is not enough: A candidate evaluation protocol for hidden-state probes of indirect prompt injection in multimodal computer-use agents," 2026, camera-ready version for the Evaluation for Multimodal Generation Workshop (EvalMG 2026). [Online]. Available: https://arxiv.org/abs/2606.22864

[7] A. Chan, C. Ezell, M. Kaufmann, K. Wei, L. Hammond, H. Bradley, E. Bluemke, N. Rajkumar, D. Krueger, N. Kolt, L. Heim, and M. Anderljung, "Visibility into AI agents," in *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. ACM, 2024, pp. 958–973.

[8] Model Context Protocol Project, "Model context protocol specification," MCP Specification, 2025. [Online]. Available: https://modelcontextprotocol.io/specification/2025-06-18/

[9] M. Sporny, A. Guy, M. Sabadello, D. Reed, O. Steele, and C. Allen, "Decentralized identifiers (DIDs) v1.0," W3C Recommendation, 2022. [Online]. Available: https://www.w3.org/TR/did-core/

[10] M. Sporny, D. Longley, and D. Chadwick, "Verifiable credentials data model v1.1," W3C Recommendation, 2022. [Online]. Available: https://www.w3.org/TR/vc-data-model/

[11] M. Kuehlewind and H. Birkholz, "An architecture for auditing AI agent delegation and interactions," Internet-Draft draft-kuehlewind-audit-architecture-00, work in progress, May 2026. [Online]. Available: https://www.ietf.org/archive/id/draft-kuehlewind-audit-architecture-00.html

[12] M. Llambí-Morillas and D. Fernández-Fernández, "Toward cryptographically verifiable authorization for autonomous AI agents: A security hypothesis, preliminary formal model, and proof-of-concept implementation," arXiv:2607.21325, 2026.

[13] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *28th USENIX Security Symposium (USENIX Security)*. USENIX Association, 2019, pp. 1393–1410.

[14] SPIFFE Project, "SPIFFE workload API," SPIFFE standard, 2026. [Online]. Available: https://spiffe.io/docs/latest/spiffe-specs/spiffe_workload_api/

[15] S. Prakash, "AIP: Agent identity protocol for verifiable delegation across MCP and A2A," arXiv:2603.24775, 2026.

[16] A. Dalugoda, "HDP: A lightweight cryptographic protocol for human delegation provenance in agentic AI systems," arXiv:2604.04522, 2026.

[17] C. Ouyang and R. Hou, "MemLineage: Lineage-guided enforcement for LLM agent memory," arXiv:2605.14421, 2026.

[18] G. Racioppi, "Mandato: Protocol-level enforcement of digitally signed mandates on AI agent actions with cryptographically chained audit trails," arXiv:2608.14074, 2026.

[19] A. C. Myers and B. Liskov, "A decentralized model for information flow control," in *Proceedings of the 16th ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 1997, pp. 129–142.

[20] M. B. Jones, J. Bradley, and N. Sakimura, "JSON web token (JWT)," IETF RFC 7519, 2015.

[21] D. Hardt, "The OAuth 2.0 authorization framework," IETF RFC 6749, 2012.

[22] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B. Yang, "High-speed high-security signatures," *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.

[23] A. Rundgren, B. Jordan, and S. Erdtman, "JSON canonicalization scheme (JCS)," IETF RFC 8785, 2020.

[24] R. C. Merkle, “Protocols for public key cryptosystems,” in *Proceedings of the 1980 IEEE Symposium on Security and Privacy*. IEEE, 1980, pp. 122–134.